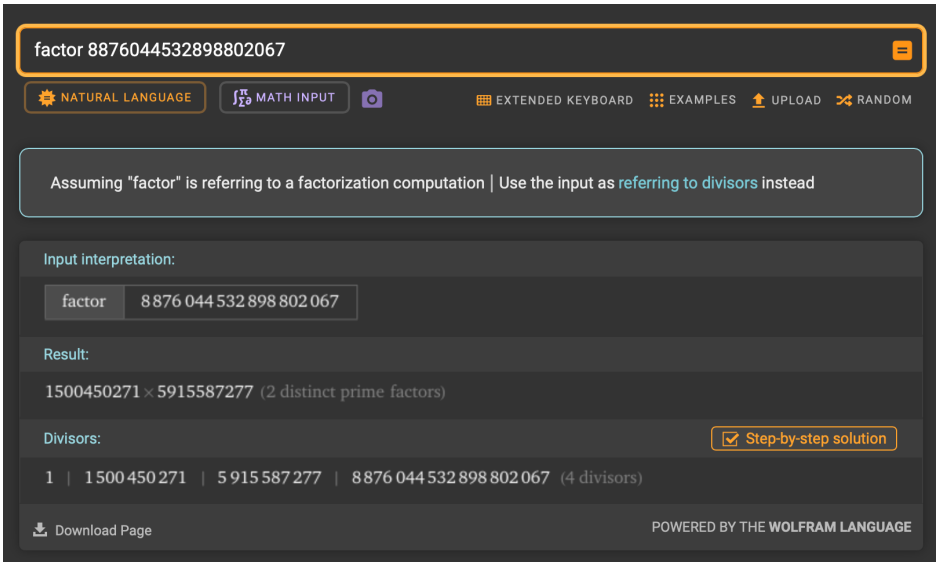
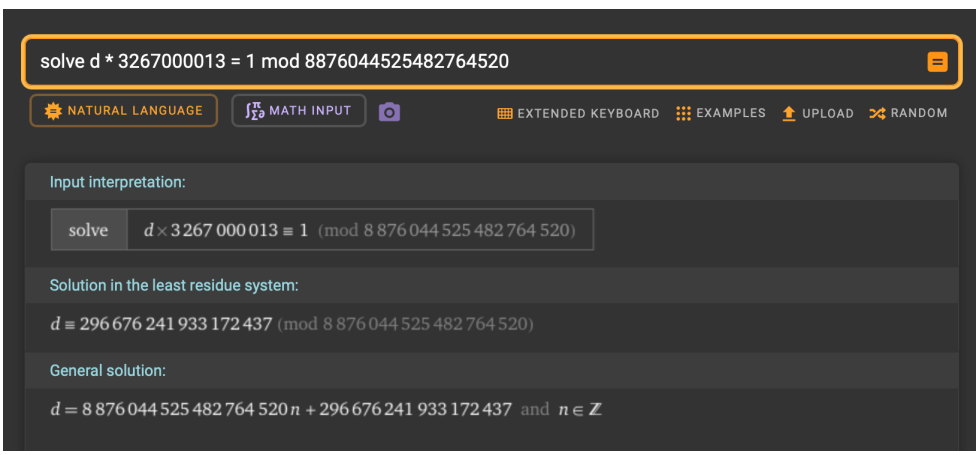
HW 5

1. RSA Decryption:
    a. n = 876044532898802067
    b. Using Wolfram Alpha: q = 1500450271, p = 5915587277
    c. 

    d. If d is the private key, then (d * e) will be congruent to 1 mod (p-1) * (q-1)
    e. (p-1) * (q-1) = 1500450270* 5915587276 = 8876044525482764520
    f. 

    g. d = 296676241933172437
    h. Encrypted message = 4504511121101132917
    i. Decrypted Message = m^d mod n
    j. 4504511121101132917^296676241933172437 mod 8876044532898802067
    k. Decrypted Message = 4242424242424242424 or BBBBBBBBB@ when converted from hex to text
2. Problem 1

a.

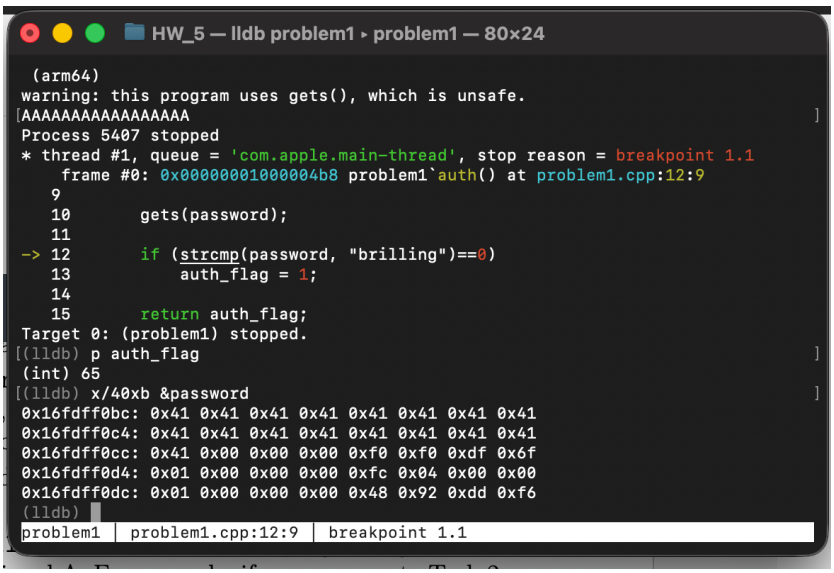
13	12	11	10	f	e	d	c	b	a	9	8	7	6	5	4	3	2	1	0
password																auth_flag			

b. int auth_flag=0;

c. 17 As to achieve access granted, as password only allocates 16 bytes, inputting the 17th A will cause it to overflow into the adjacent memory address, which causes the security to fail.

d. 16 bytes between auth_flag and password memory addresses.
   i. auth_flag address: 0x000000016fdff0cc
   ii. password address: 0x000000016fdff0bc
      1. cc – bc = 10 (16 in base 10)

e. 4 bytes needed to store auth_flag, with a value of 65 when AAAAAAAAAAAAAAAA is entered.

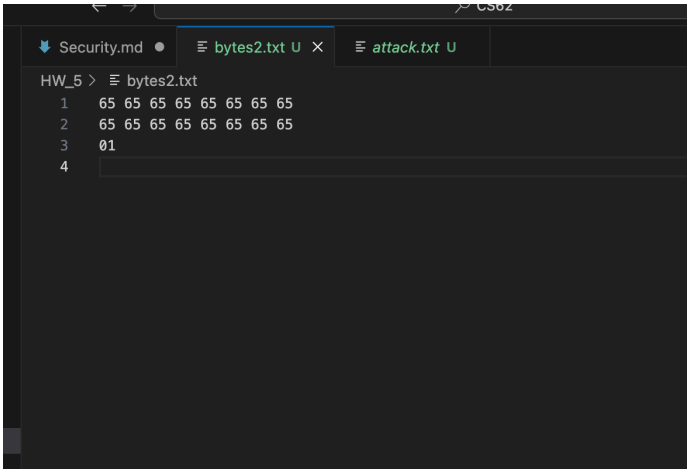
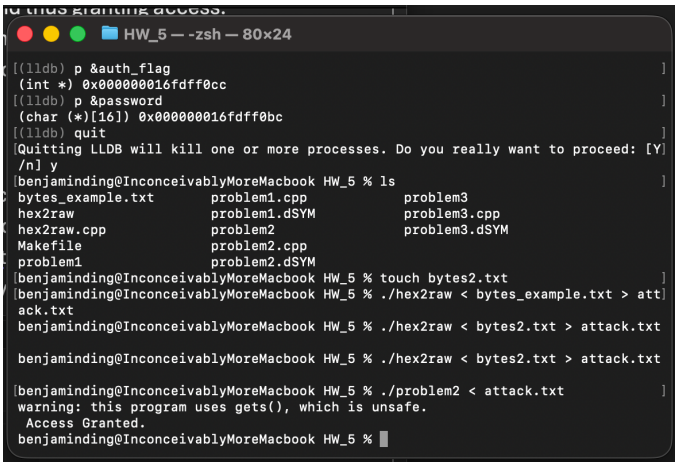
f. 

g. Because password was only allocated 16 bytes, the memory address was placed 16 bytes above auth_flag. So when a 17 character string is input into password, the 17th character is written at auth_flag's memory address, overriding the initialized value of 0, and thus granting access.

h. 16705, which is 4141 in hex. Because the memory would have been written as 65, and another 65, it would have looked like 4141 and then converted all at once back to decimal.

3. Problem 2
   a. Memory addresses are the same as problem 1
      i. auth_flag address: 0x000000016fdff0cc
      ii. password address: 0x000000016fdff0bc

b. To break this auth_flag, the 17th character has to be 01 in hex instead of A. This will be difficult because 01 is not an easily typable character

c. 

d.